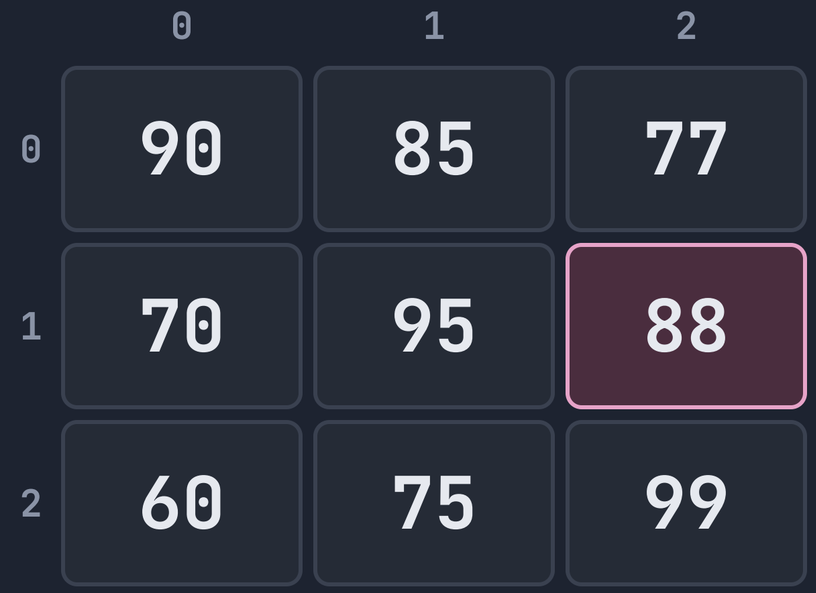
KANT AX/AI 엔지니어 트랙 · 보충 학습 자료

파이썬 기초 3
in과
2차원 리스트

들어 있는지 묻고,
표 모양 데이터 다루기



scores[1][2] → 88

오늘의 순서

1 **in과 not in**
리스트, 튜플, 문자열에서 들어 있는지 묻기, and와 or로 섞기, 한 겹만 보는 in — 4–8쪽

2 **2차원 리스트**
[행][열]로 꺼내기, 행 수와 열 수, for로 행과 열 다루기, 13의 리스트와 비교, 함정과 오류 — 9–21쪽

3 **실제로 써 보기**
성적표, 좌석표, 카페 판매표 — 22–24쪽

3쪽은 노트북 준비, 25–26쪽은 막힐 때와 한 장 요약이에요.

들어가기

13에서 이어서 해요

- ✓ hello-uv 노트북, 커널 .venv — 09
- ✓ 비교와 and, or, not — 12
- ✓ 리스트, 튜플, count, index — 13
- ✓ for 문, += 로 합계 쌓기 — 13
- ✓ f-string 칸 수 맞추기 — 13

노트북 다시 열기

1. VS Code로 hello-uv 폴더 열기
2. 탐색기에서 `basics.ipynb` 열기
3. 오른쪽 위 커널이 .venv인지 보기
4. 셀에 적고 `Shift` + `Enter` 로 실행

기억이 안 나면 줄 끝에 적힌 챕터를 먼저 다시 봐요.

1. in과 not in

count나 index로 있는지 물으면 한 번 더 따져야 해요

[1]
```python
menu = ["coffee", "latte", "cake"]
print(menu.count("juice"))
menu.index("juice")
```

```
0
ValueError: 'juice' is not in list
```

← 0개라서 없다는 뜻이에요

← 없으면 오류로 멈춰요

있는지만 묻고 싶을 때는 이렇게 써요

```
"juice" in menu → False
```

1. in과 not in

in은 들어 있으면 True, 없으면 False예요

[2]
```python
print("latte" in menu)
print("juice" in menu)
print("juice" not in menu)
print("latte" not in menu)
```

```
True    ← latte가 있어요
False   ← juice는 없어요
True    ← 없는 게 맞아요
False   ← 있으니까 틀렸어요
```



`in`	들어 있으면 True
`not in`	없으면 True

1. in과 not in

튜플과 문자열에도 in을 써요

[3]
```python
order = ("coffee", 4500, 2)
print(4500 in order)
print("4500" in order)
```

```
True
False   ← 글자 "4500"은 숫자 4500과 다른 값이에요
```

order

0	1	2
"coffee"	4500	2

[4]
```python
word = "python"
print("th" in word)
print("pt" in word)
print("P" in word)
```

```
True    ← th는 붙어 있어요
False   ← p와 t는 떨어져 있어요
False   ← 대문자와 소문자를 구분해요
```

word

0	1	2	3	4	5
p	y	t	h	o	n

p와 t 사이에 y가 있어요

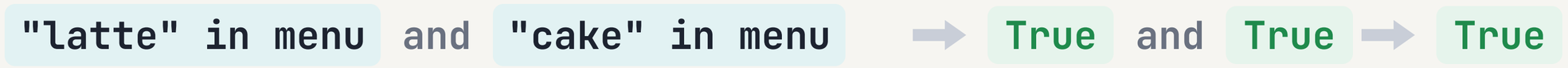
1. in과 not in

in의 답은 and, or, not과 섞어 써요

[5]
```python
has_both = "latte" in menu and "cake" in menu
print(has_both)
print("juice" in menu or "tea" in menu)
print(not "juice" in menu)
```

```
True    ← 둘 다 있어서 True
False   ← juice도 tea도 없어서 False
True    ← "juice" not in menu와 같아요. not in 쪽이 읽기 쉬워요
```

has_both 줄은 이렇게 계산돼요



① in 두 개를 먼저 계산해요

② and는 그다음

1. in과 not in

in은 바깥 한 겹만 들여다봐요

[6]
```python
orders = [("coffee", 4500, 2), ("latte", 5000, 1), ("cake", 6500, 3)]
print(("cake", 6500, 3) in orders)
print("cake" in orders)
print("cake" in orders[2])
```

```
True    ← 튜플 통째로 물으면 찾아요
False   ← 바깥에는 튜플 세 개뿐이라 글자 cake는 없어요
True    ← orders[2]로 한 겹 꺼내면 안에 있어요
```

orders 바깥 리스트



안쪽까지 보려면 번호로 한 겹 꺼낸 뒤 in을 써요.

2. 2차원 리스트

학생마다 변수를 따로 두면 학생 수만큼 코드가 늘어요

[1]
```python
kim = [90, 85, 77]
lee = [70, 95, 88]
park = [60, 75, 99]
print(sum(kim) + sum(lee) + sum(park))
```

```
739
```

← 학생 한 명에 변수 하나

← 30명이면 sum을 30번 적어요

scores

90	85	77	← kim
70	95	88	← lee
60	75	99	← park

세 줄을 리스트 하나에 담으면
for로 한 번에 돌 수 있어요

2. 2차원 리스트

리스트 안에 리스트를 넣으면 표가 돼요

```
[2] scores = [[90, 85, 77],
              [70, 95, 88],
              [60, 75, 99]]
    print(scores)
    print(len(scores))
    print(len(scores[0]))
```
Python

```
[[90, 85, 77], [70, 95, 88], [60, 75, 99]]
3
3
```

← len(scores): 바깥 리스트에 든 행 수

← len(scores[0]): 0행 안의 칸 수가 열 수

	0 kor	1 eng	2 math
0	90	85	77
1	70	95	88
2	60	75	99

가로 한 줄이 행이에요. 안쪽 리스트 하나가 한 행

세로 한 줄이 열이에요. 과목 하나가 한 열

대괄호가 닫히기 전에는 줄을 바꿔 적어도 돼요. 그래서 표처럼 적어요.

2. 2차원 리스트

[행][열] 순서로 칸 하나를 꺼내요

[3]
```python
print(scores[1])
print(scores[1][2])
print(scores[0][0])
print(scores[-1][-1])
```

```
[70, 95, 88]   ← 1행 통째로
88   ← 1행의 2열
90   ← 맨 왼쪽 위
99   ← 마지막 행의 마지막 열
```

	0	1	2
0	90	85	77
1	70	95	88
2	60	75	99

앞 번호가 행, 뒤 번호가 열이에요. 둘 다 0부터 세요.

번호 두 개면 칸 하나, 번호 하나면 행 하나를 바꿔요

[4]
```python
scores[0][1] = 100  ← 칸 하나
print(scores[0])
scores[2] = [65, 80, 90]  ← 행 하나
print(scores)
```

```
[90, 100, 77]
[[90, 100, 77], [70, 95, 88], [65, 80, 90]]
```

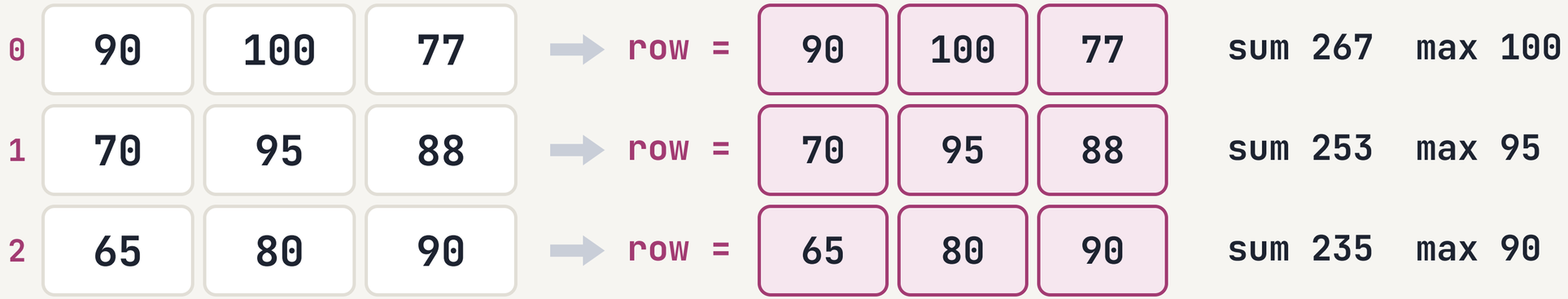
	0	1	2
0	90	100	77
1	70	95	88
2	65	80	90

다음 장부터는 이렇게 바뀐 scores를 써요.

for로 한 행씩 꺼내면 row에는 리스트가 들어가요

[5]
```python
for row in scores:
    print(row, "합계:", sum(row), "최고:", max(row))
```

```
[90, 100, 77] 합계: 267 최고: 100
[70, 95, 88] 합계: 253 최고: 95
[65, 80, 90] 합계: 235 최고: 90
```



row가 리스트라서 13에서 배운 sum, max를 바로 써요.

열 하나는 행마다 같은 번호의 칸을 모아요

[6]
```python
eng = []
for row in scores:
    eng.append(row[1])
print(eng)
print("영어 합계:", sum(eng))
```

```
[100, 95, 80]
영어 합계: 275
```

← 모은 eng는 보통 리스트라 sum을 써요

for가 돌 때마다 row[1]이 하나씩 들어가요

eng

100	95	80

	0 kor	1 eng	2 math
0	90	100	77
1	70	95	88
2	65	80	90

행은 scores[1]로 바로 꺼내지만, 열은 for로 모아요.

표 전체 합계는 행 합계를 차례로 쌓아요

[7]
```python
total = 0
count = 0
for row in scores:
    total += sum(row)
    count += len(row)
print("전체 합계:", total)
print("칸 수:", count)
print(f"평균: {total / count:.1f}")
```

```
전체 합계: 755
칸 수: 9
평균: 83.9
```

for가 한 번 돌 때마다

row	sum(row)	total	count
[90, 100, 77]	267	267	3
[70, 95, 88]	253	520	6
[65, 80, 90]	235	755	9

755 / 9 = 83.888...

:.1f 로 소수점 한 자리만 보여 줘요 (13)

total과 count는 for 앞에서 0으로 한 번만 시작해요.

자르기도 앞 번호는 행, 뒤 번호는 열에 써요

[8]
```python
print(scores[:2])
print(scores[1][:2])
print(scores[0][::-1])
```

```
[[90, 100, 77], [70, 95, 88]]
[70, 95]
[77, 100, 90]
```

[시작:끝:간격]은 13에서 배운 그대로예요.
어느 리스트를 자르는지만 봐요.

`scores[:2]`

바깥을 잘라서 행 두 개

90	100	77
70	95	88
65	80	90

`scores[1][:2]`

1행을 꺼낸 뒤 칸 두 개

90	100	77
70	95	88
65	80	90

`scores[0][::-1]`

0행을 거꾸로

90	100	77
70	95	88
65	80	90

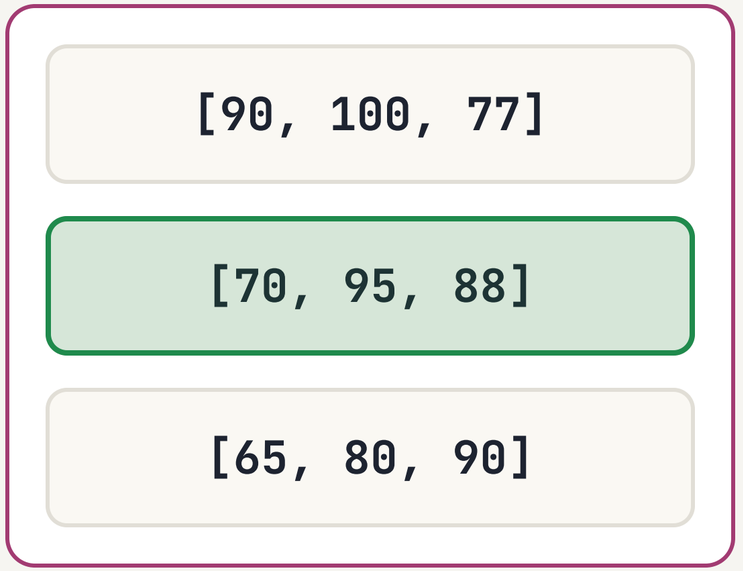
열 하나를 통째로 자르는 문법은 없어요. 앞 장처럼 for로 모아요.

2차원 리스트에서 in은 바깥의 행만 봐요

[9]
```python
print(95 in scores)
print(95 in scores[1])
print([70, 95, 88] in scores)
for row in scores:
    print(row, 100 in row)
```

```
False   ← 바깥에는 행 리스트 세 개만 있어요
True    ← 1행을 꺼내서 물으면 찾아요
True    ← 행을 통째로 적으면 찾아요
[90, 100, 77] True   ← for로 행마다 물어요
[70, 95, 88] False
[65, 80, 90] False
```

scores 바깥 리스트



1파트의 orders와 같은 모양이에요

행은 append 한 번, 열은 행마다 append해요

[10]
```python
scores.append([80, 70, 60])
print(len(scores))
for row in scores:
    row.append(0)
for row in scores:
    print(row)
```

```
4
[90, 100, 77, 0]
[70, 95, 88, 0]
[65, 80, 90, 0]
[80, 70, 60, 0]
```

	0	1	2	3
0	90	100	77	0
1	70	95	88	0
2	65	80	90	0
3	80	70	60	0

새 행은 바깥 리스트에 하나

새 열은 행 네 개에 하나씩

row는 scores 안의 행 그 자체라서, row에 붙이면 scores도 바뀌어요.

2. 2차원 리스트

번호가 하나 늘었을 뿐, 13의 리스트와 쓰는 법이 같아요

	리스트 (13)	2차원 리스트 (14)
만들기	`[90, 85, 77]`	`[[90, 85, 77], ...]`
칸 꺼내기	`xs[1]`	`scores[1][2]`
개수	`len(xs)`	`len(scores)` 행 수
for	`for x in xs:`	`for row in scores:` row는 행
더하기	`xs.append(5)`	`scores.append([80, 70, 60])`
in	`85 in xs`	`85 in scores[0]`
자르기	`xs[:2]`	`scores[:2]` `scores[1][:2]`

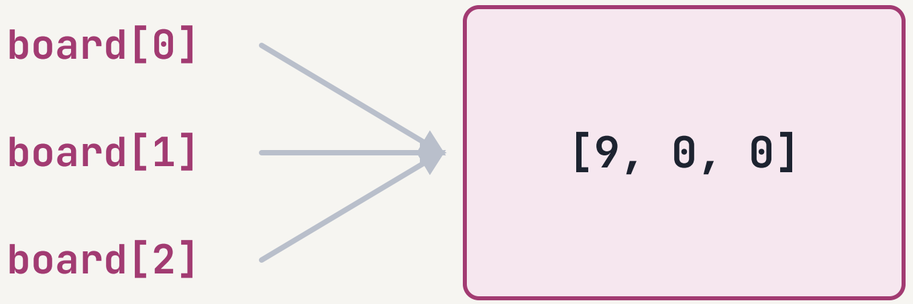
안쪽까지 가려면 번호를 하나 더 붙여요.

[[0] * 3] * 3은 같은 행 하나를 세 번 담아요

[11]
```python
board = [[0] * 3] * 3
board[0][0] = 9
print(board)
```

```
[[9, 0, 0], [9, 0, 0], [9, 0, 0]]
```

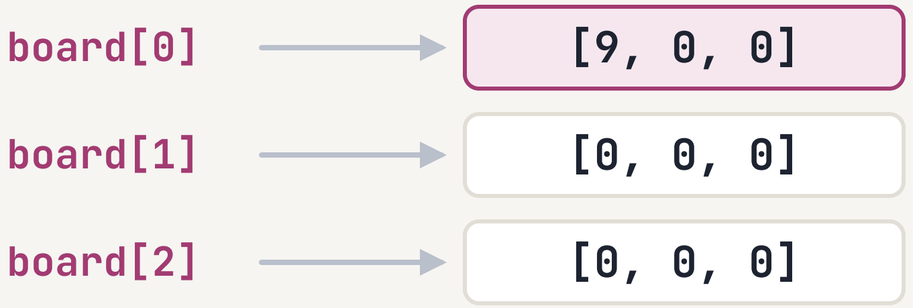
행 리스트 하나를 셋이 같이 봐요



[12]
```python
board = [[0, 0, 0],
         [0, 0, 0],
         [0, 0, 0]]
board[0][0] = 9
print(board)
```

```
[[9, 0, 0], [0, 0, 0], [0, 0, 0]]
```

행이 세 개 따로 있어요



빈 표는 행을 하나씩 직접 적어서 만들어요.

2. 2차원 리스트

번호가 넘치면 IndexError, 개수가 다르면 ValueError예요

[13]
```
scores[5][0]
```
Python

```
IndexError: list index out of range
```

← 행은 0~3번, 네 개뿐이에요

[14]
```
scores[0][9]
```
Python

```
IndexError: list index out of range
```

← 0행 안의 칸도 0~3번이에요

[15]
```
kor, eng, math = scores[0]
```
Python

```
ValueError: too many values to unpack (expected 3)
```

← 한 행에 값이 네 개예요

확인하기 `len(scores)` 행 수 4 `len(scores[0])` 열 수 4

두 IndexError는 메시지가 같아요. 앞 번호와 뒤 번호 중 어느 쪽이 넘쳤는지 len으로 봐요.

3. 실제로 써 보기

이름을 넣은 성적표를 칸 맞춰 찍어요

[1]
```python
students = [["kim", 90, 85, 77],
            ["lee", 70, 95, 88],
            ["park", 60, 75, 99]]
print(f"{'name':<6}{'kor':>5}{'eng':>5}{'math':>5}")
print("-" * 21)
for name, kor, eng, math in students:
    print(f"{name:<6}{kor:>5}{eng:>5}{math:>5}")
```

Python

← 행 맨 앞에 이름을 넣었어요

← 한 행의 값 네 개를 변수 네 개에

← :>5 는 오른쪽 맞춤 다섯 칸 (13)

```
name    kor  eng math
---------------------
kim      90   85   77
lee      70   95   88
park     60   75   99
```

학생이 늘어도 for 두 줄은 그대로예요

for 뒤 변수 개수가 행의 값 개수와 같아야 해요. 다르면 ValueError예요.

3. 실제로 써 보기

좌석표도 행과 열이 있는 2차원 리스트예요

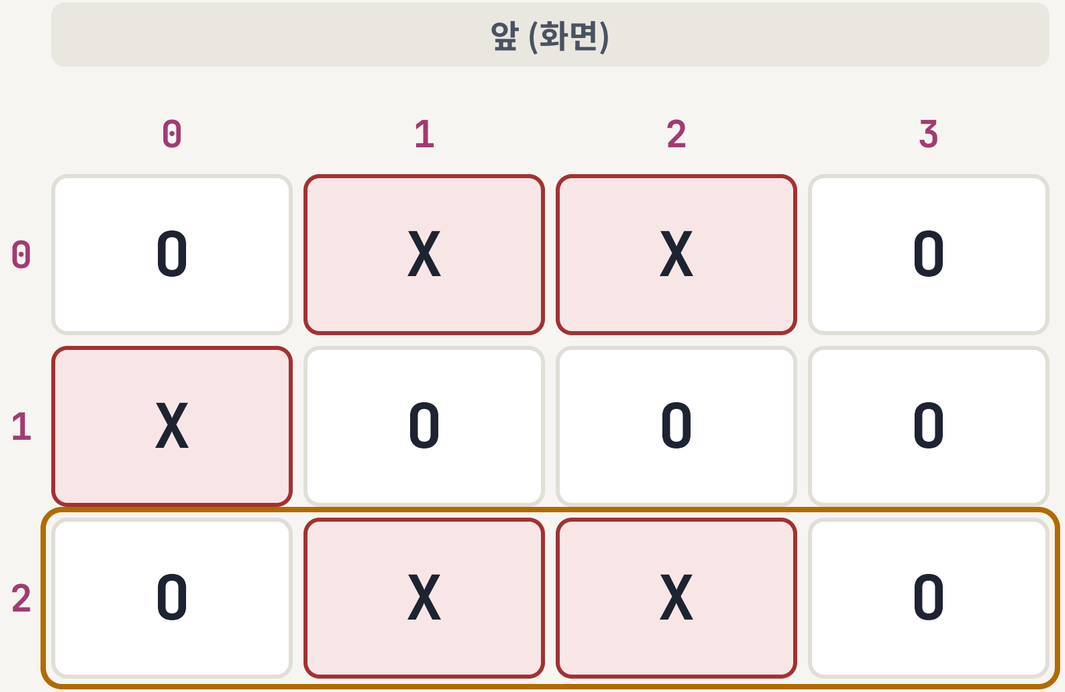
[2]
```
seats = [["O", "O", "X", "O"],
         ["X", "O", "O", "O"],
         ["O", "X", "X", "O"]]
seats[0][1] = "X"
for row in seats:
    print(" ".join(row))
print(seats[2].count("X"))
print(len(seats) * len(seats[0]))
```
Python

```
O X X O
X O O O
O X X O
2
12
```

← join으로 행마다 한 줄 (13)

← 2행의 찬 자리 2개

← 3행 × 4열 = 전체 12석



3. 실제로 써 보기

카페 판매표에서 하루 합계와 메뉴 합계를 구해요

[3]
```python
menu = ["coffee", "latte", "cake"]
sales = [[12, 8, 5],
         [10, 9, 7],
         [15, 11, 4]]
for row in sales:
    print(row, "하루 합계:", sum(row))
col = menu.index("latte")   ← latte는 1열
latte = []
for row in sales:
    latte.append(row[col])
print("latte:", latte, "합계:", sum(latte))
```

```
[12, 8, 5] 하루 합계: 25
[10, 9, 7] 하루 합계: 26
[15, 11, 4] 하루 합계: 30
latte: [8, 9, 11] 합계: 28
```

	0 coffee	1 latte	2 cake	합계
0 월	12	8	5	25
1 화	10	9	7	26
2 수	15	11	4	30

latte 열을 모으면 [8, 9, 11], 합계 28

정리

막히면 오류의 마지막 줄을 봐요

1	`TypeError: 'in <string>' requires string as left operand, not int` 문자열 안에 숫자를 물었어요	`"3" in word`
2	`IndexError: list index out of range` 행이나 열 번호가 넘쳤어요	len()으로 확인
3	`ValueError: too many values to unpack (expected 3)` 행의 값 개수와 변수 개수가 달라요	변수를 값 수만큼
4	`[[9, 0, 0], [9, 0, 0], [9, 0, 0]]` 칸 하나만 바꿨는데 모든 행이 바뀌었어요	행을 직접 적기
5	`95 in scores → False` in은 바깥 한 겹만 봐요	`95 in scores[1]`
6	`"P" in word → False` 대문자와 소문자가 달라요	`"p" in word`

오류가 안 나도 결과가 이상하면 몇 겹을 보고 있는지 따져 봐요.

한 장 요약

in과 not in

`"latte" in menu`	들어 있으면 True
`"tea" not in menu`	없으면 True
`"th" in word`	붙어 있는 글자 조각
`a in xs and b in xs`	둘 다 들어 있는지

2차원 리스트

`scores[1][2]`	1행 2열 칸
`len(scores[0])`	열 수
`for row in scores:`	한 행씩 꺼내기
`col.append(row[1])`	열 하나 모으기

in은 바깥 한 겹만 봐요. 안쪽은 번호로 꺼낸 뒤 물어요.
`[행][열]` 순서예요. 행 수는 len(scores), 열 수는 len(scores[0])예요.

다음 챕터 15에서는 번호 대신 키로 값을 꺼내는 딕셔너리를 배워요.